

Research

A novel lightweight CNN design for MRI brain tumor image classification with performance-driven optimization

G. Appasami¹ · Nickolas Savarimuthu¹

Received: 7 January 2025 / Accepted: 11 May 2025

Published online: 27 September 2025

© The Author(s) 2025 **OPEN**

Abstract

Accurately classifying brain tumors is a crucial task in medical imaging analysis, with significant implications for diagnosis and treatment planning. Early detection of brain tumors helps to save lives and ensures individuals remain in a safe zone. This research paper presents a novel lightweight custom-built Convolutional Neural Network (CNN) architecture optimized for brain tumor classification and is compared with standard pre-trained CNN architectures like VGG, Xception, and ResNet. It explores the impact of different Convolutional kernel sizes, max pooling strategies, batch sizes, and cross-dataset validation on model performance. A series of CNN models built with valid kernel size, pooling size, and batch sizes were trained and evaluated with performance metrics, including accuracy, precision, recall, and F1-score, which were analyzed across these different configurations. Additionally, five cross-dataset validations were conducted to assess model generalization. The results indicate that increasing kernel sizes and batch sizes does not improve model performance. The optimal configuration of a 4×4 Convolutional kernel, 4×4 max pooling size, and a batch size of 64 achieved the highest performance, with an accuracy of 99.54% and strong precision and recall metrics across all datasets. The proposed lightweight model, with a compact size of just 6.89 MB and only 1.8 million parameters, features a streamlined architecture of just nine layers while maintaining high accuracy. New findings suggest that efficiency in lightweight model design can be achieved with high classification accuracy, providing valuable insights for medical image analysis.

Keywords MRI brain tumor image classification · Lightweight convolutional neural network · Medical image analysis · Deep learning · CNN model efficiency · Kernel and pooling optimization

1 Introduction

Brain tumor classification using Magnetic Resonance Imaging (MRI) is a pivotal area of medical image analysis, offering significant potential to assist clinicians in the early detection, diagnosis, and treatment planning of brain tumors. The accurate classification of brain tumors plays an essential role in improving patient outcomes by enabling timely interventions. Traditional methods of tumor detection, which often rely on manual interpretation by radiologists or rule-based algorithms, are susceptible to human error and can be time-consuming. However, with the rapid advancement of deep learning techniques like Convolutional Neural Networks (CNNs) have emerged for automating the classification of medical images, including brain MRI scans [1]. CNNs excel in extracting hierarchical spatial features from raw image data, eliminating the need for manual feature engineering, and have shown remarkable performance in various medical imaging applications, particularly brain tumor classification [2, 3].

✉ G. Appasami, appa.govi@gmail.com; Nickolas Savarimuthu, nickolas@nitt.edu | ¹National Institute of Technology Tiruchirappalli, Tiruchirappalli, Tamilnadu, India.



Despite their successes, CNNs come with challenges, particularly in terms of their high computational demands. Many advanced architectures, such as ResNet, VGG, and Xception, are large and require substantial memory and processing power, which can be a limitation in resource-constrained environments like mobile devices or real-time clinical settings. These constraints hinder the applicability of deep learning models in environments where quick processing is essential for real-time decision-making. To address these issues, recent research has focused on optimizing CNN architectures to balance accuracy with computational efficiency [4]. Lightweight models that retain high classification accuracy while significantly reducing computational requirements have been developed to meet the needs of clinical environments where real-time processing is crucial [5]. This trend towards lightweight models has opened up new possibilities for deploying deep learning-based diagnostic tools on platforms with limited resources [6].

Another challenge in brain tumor classification is the optimization of CNN hyperparameters, such as kernel (filter) sizes, pooling strategies, and batch sizes. The choice of these parameters can significantly impact model performance. For instance, smaller kernels capture finer details in images, while larger kernels tend to capture broader features. Max pooling layers, which are essential for reducing spatial dimensions, also affect the network's ability to retain important features [7]. Furthermore, optimizing batch size is critical for the model's convergence and stability during training. The role of cross-dataset evaluation is equally important, as it helps to ensure that the model is robust and can be generalized across different MRI datasets. This aspect is vital because variations in image quality, acquisition methods, and patient demographics can influence the model's performance across different clinical environments [8, 9].

The application of CNNs in brain tumor classification is further enhanced by exploring hybrid models that combine CNNs with other machine-learning techniques. These hybrid models aim to leverage the strengths of both CNNs and traditional machine learning methods to improve classification accuracy and model robustness [10, 11]. Additionally, multi-scale architectures and multi-modal data integration are being increasingly explored to capture various levels of information and enhance the model's ability to handle complex and diverse data [12]. The goal is to create models that are not only accurate but also efficient, generalizable, and capable of handling various clinical challenges [13, 14].

This paper reviews recent advancements in deep learning-based brain tumor classification, with a particular focus on the development of lightweight CNN models that maintain high performance while addressing computational efficiency. The paper also explores emerging hybrid models, multi-scale architectures, and strategies for integrating multi-modal data to create more robust models. By reviewing the challenges faced in the field, including the need for high-quality datasets and improved model generalization, this paper outlines the potential for designing effective models that can be deployed in real-time clinical settings, ultimately contributing to more accessible and reliable brain tumor diagnosis [15, 16].

The paper is structured as follows: Sect. 1 introduces the research, outlining the problem, objectives, and motivation behind the research. Section 2 reviews related work, summarizing existing research and methodologies. Section 3 details the proposed methodology, including datasets, model architecture, and experimental setup. Section 4 presents results and analysis, comparing findings with existing solutions. Finally, Sect. 5 concludes by summarizing the paper's contributions, directions for future work, and potential applications..

2 Related work

Brain tumor classification from MRI images has become a pivotal area of research in medical image analysis, addressing the urgent need for accurate, efficient, and automated diagnostic tools. Traditional methods for tumor detection have often relied on manual interpretation by radiologists or conventional image processing techniques, both of which are time-consuming and prone to human error. As these methods are increasingly replaced by advanced deep learning models, particularly Convolutional Neural Networks (CNNs), the field has seen significant improvements. CNNs, due to their ability to automate image analysis, have proven highly effective in classifying brain tumors. Gong et al. [1] provide a comprehensive survey on the application of deep learning techniques in brain tumor classification, highlighting the advantages of CNNs in automatically extracting relevant features from raw MRI data. This deep learning approach removes the need for manual feature extraction, allowing CNNs to learn hierarchical patterns that improve classification accuracy and efficiency.

Despite the promising performance of CNNs, a significant challenge lies in balancing high performance with computational efficiency. Traditional CNN architectures like VGG16 and ResNet, while achieving outstanding results, are computationally expensive and require substantial memory and storage. For example, VGG16 with 138.4 million parameters

occupies 528 MB of storage, making it impractical for real-time applications, especially in resource-constrained environments. Zhou et al. [5] address this challenge by proposing a lightweight CNN model with only 1.3 million parameters while achieving a 96.67% classification accuracy. This lightweight model significantly reduces computational overhead, making it suitable for deployment in mobile or edge computing environments, where resources are limited.

In recent years, hybrid deep learning models have gained attraction for enhancing tumor classification performance by combining CNNs with other techniques. Ahamed et al. [6] introduced a hybrid model integrating CNNs with recurrent neural networks (RNNs) to capture temporal features in sequential MRI scans, such as pre-and post-treatment images. This model demonstrated superior classification performance by utilizing both spatial and temporal information, which improved the accuracy of tumor detection [17]. Similarly, Raza et al. [18] explored multi-modal MRI data, combining T1-weighted, T2-weighted, and FLAIR scans to improve classification accuracy. By incorporating multiple modalities, their model extracted a broader range of features, improving tumor differentiation and classification.

Architectural innovations focused on improving feature extraction through multi-scale and feature fusion strategies have also gained attention. Wu et al. [9] proposed a multi-scale CNN architecture that captures features at different granular levels, from fine details to broader spatial patterns. This approach enables better preservation of subtle and large-scale tumor characteristics, thus improving classification accuracy. Thapliyal et al. [19] investigated feature fusion strategies, which combine low-level and high-level features from different CNN layers to improve classification performance. This technique has been shown to provide a richer set of features, enhancing the model's ability to distinguish between various tumor types.

Enhancing the generalization ability and robustness of CNNs remains a key focus in recent research. Singh et al. [13] introduced a multi-layer CNN architecture that incorporates feature fusion strategies, which significantly improved the classification of benign and malignant tumors. This architecture's ability to generalize across different tumor types underscores the importance of using diverse data sources in training. Saeed et al. [14] emphasized the need for high-quality annotated datasets and multi-modal data integration to build more robust CNN models. Their work highlights the challenges of dealing with imbalanced datasets, which are common in medical imaging, and the importance of developing models that generalize well to unseen MRI data, ensuring reliable performance in clinical settings.

While models such as Xception, ResNet, and VGG16 have demonstrated excellent results in medical imaging tasks, their computational demands present challenges for real-world deployment. Xception, for example, requires 88 MB of storage and has 22.9 million parameters, making it impractical for low-resource environments. Similarly, ResNet152 and InceptionV3 require substantial memory and processing power, which makes them unsuitable for edge computing or real-time applications. Researchers have explored techniques such as pruning, knowledge distillation, and model quantization to reduce model complexity while maintaining performance. However, these approaches often come with trade-offs in accuracy, as observed by authors like Li et al. [20] and Chen & Zhao [21].

Despite the significant progress made in CNN architectures for brain tumor classification, several research gaps remain. First, while studies have explored different CNN architectures, there is limited analysis of how variations in kernel sizes and pooling layers impact the overall performance of the model, particularly in terms of accuracy and computational efficiency across diverse MRI datasets [1, 2, 16]. Second, while CNN architectures have advanced, the impact of batch sizes on training dynamics and model generalization is under-explored, especially when dealing with imbalanced datasets, which are common in medical imaging [5, 18]. Additionally, while cross-dataset evaluations have been performed, results remain inconsistent due to differences in MRI data characteristics [20, 21]. There is a need for standardized protocols for cross-dataset validation to ensure the robustness of CNN models. Lastly, while individual CNN components have been optimized, hybrid approaches combining CNN with other deep learning models, such as RNNs, remain under exploration [18]. Future research should explore the synergistic effects of combining various model components, hyperparameter tuning, and cross-dataset validation to improve the clinical applicability of brain tumor classification systems.

In CNNs, two key hyperparameters that need optimization are kernel size and pooling. A kernel (or filter) is a small matrix of weights that slides over the input image or feature map to perform convolution, enabling the network to extract features such as edges, textures, and shapes. For instance, a 3×3 kernel captures small, localized patterns, while larger kernels (e.g., 5×5 or 7×7) capture broader features [15]. Pooling is a downsampling technique that reduces the spatial dimensions of feature maps, decreasing computational cost and preventing overfitting. The most common types of pooling are max pooling, where the maximum value from a region is selected, and average pooling, which computes the average value [16]. For example, a 2×2 max pooling operation selects the maximum value from each 2×2 region, reducing the feature map's size [7]. Optimizing kernel size and pooling strategy is essential for improving CNN performance by balancing feature extraction and computational efficiency, resulting in better generalization and faster training.

The optimization process involves configuring essential parameters such as kernel sizes, learning rates, dense layer sizes, and dropout rates, learning rate multiplying factor, while utilizing optimizers like Adam and callbacks for dynamic learning rate adjustments and enhanced training efficiency. Standardized metrics and strategies are useful to improve and evaluate CNN performance. The proposed work presents a lightweight CNN architecture optimized for clinical deployment, balancing model size, computational efficiency, and classification performance. Key hyperparameters, including kernel sizes, pooling strategies, and batch sizes, are fine-tuned to achieve high accuracy while minimizing computational overhead, making the model suitable for mobile and edge platforms. Cross-dataset validation further ensures the model's robustness across diverse MRI datasets.

3 Methodology

The proposed model was designed to achieve high classification accuracy while minimizing the computational footprint. It consists of only 9 layers and has 1.3 million parameters, making it highly efficient in terms of both storage and inference time [22]. The model was evaluated on a variety of brain tumor datasets, including labeled MRI brain tumor datasets and cleaned MRI tumor data, and compared against several state-of-the-art CNN architectures such as Xception, VGG16, ResNet, DenseNet, and EfficientNet [1].

3.1 Datasets

The primary dataset, Dataset 0, located at 'kaggle/input/brain-tumor-mri-dataset', serves as the foundation for training and optimizing the model's hyperparameters. This image dataset consists of a total of 7,023 MRI images of human brains, systematically classified into four classes: glioma, meningioma, no tumor, and pituitary tumor. The training set comprises 5,712 digital MRI images of the human brain and is distributed across the classes as follows: glioma has 1,321 images, meningioma contains 1,339 images, no tumor consists of 1,595 images, and pituitary tumor has 1,457 images. In contrast, the testing set contains 1,311 images, which are allocated as follows: glioma has 300 images, meningioma contains 306 images, no tumor consists of 405 images, and pituitary tumor has 300 images [16].

The classification model is used to evaluate using five diverse MRI brain tumor datasets for cross-dataset validation, each consisting of four tumor categories: glioma, meningioma, no tumor, and pituitary tumor. Dataset 1, the Labeled MRI Brain Tumor Dataset, contains 2175 images and is located at kaggle/input/labeled-mri-brain-tumor-dataset [15]. Dataset 2, the Cleaned MRI Brain Tumor Dataset, includes 5099 images and is available at kaggle/input/mri-brain-tumor-cleaned [21]. Dataset 3, the Code-to-Cure Dataset, has 5229 images and can be found at the following link kaggle/input/code-to-cure [23]. Dataset 4, the MRI Brain Tumor Classification Dataset, includes 5708 images and is accessible at kaggle/input/mri-brain-tumor-classification [24]. Finally, Dataset 5, the Brain Tumor MRI Dataset contains 5712 images and is located at kaggle/input/brain-tumor-mri-dataset-final [20]. These five datasets collectively provide a comprehensive framework for cross-dataset evaluation as shown in Table 1, ensuring the model's robustness and generalization [21].

The complete dataset can be freely accessed on Kaggle, making it a rich resource for researchers and practitioners in the fields of medical imaging and deep learning. Its comprehensive nature facilitates various experiments in the classification of brain tumors, contributing significantly to advancements in medical diagnostics methodologies [25]. The dataset's accessibility enhances collaborative research efforts and supports the development of innovative solutions in the medical imaging domain [22].

Table 1 MRI brain tumor classification dataset

Dataset	Purpose	Images	Contributor	Code link	Website link
Dataset 0	Training and Testing	7,023	Masoud Nickparvar	kaggle/input/brain-tumor-mri-dataset	https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset
Dataset 1	Cross validation	2,175	Ammar Ahmed	kaggle/input/labeled-mri-brain-tumor-dataset	https://www.kaggle.com/datasets/ahmed310/labeled-mri-brain-tumor-dataset
Dataset 2	Cross validation	5,099	Anna Shahinyan	kaggle/input/mri-brain-tumor-cleaned	https://www.kaggle.com/datasets/annafivebrane/mri-brain-tumor-cleaned
Dataset 3	Cross validation	5,229	Jaadarix	kaggle/input/code-to-cure	https://www.kaggle.com/datasets/jaadari/code-to-cure
Dataset 4	Cross validation	5,708	Rijul Raman	kaggle/input/mri-brain-tumor-classification	https://www.kaggle.com/datasets/rijulraman/mri-brain-tumor-classification
Dataset 5	Cross validation	5,712	Ujjwal Jaiswal	kaggle/input/brain-tumor-mri-dataset-final	https://www.kaggle.com/datasets/ujjwaljaiswal9/brain-tumor-mri-dataset-final

3.2 Algorithm

The MRI Brain Tumor Image Classification algorithm 1 presents a structured methodology to optimize the architecture of Convolutional Neural Networks for the classification of MRI brain tumor images [1]. Step 1 of the algorithm begins with initializing essential parameters such as Convolutional kernel sizes, pooling sizes, batch size, learning rates, and dropout rates. It also incorporates advanced mechanisms like adaptive learning rate reduction and early stopping to enhance the training process [5]. The parameter choices aim to explore various potential configurations while maintaining efficient computational resource usage. The datasets [D0, D1, ..., D5] are designated for training and cross-validation to ensure the robustness of the model [18].

Step 2 evaluates different combinations of kernel and pooling sizes by building models for each combination and training them on dataset D0. The use of callbacks for dynamic learning rate adjustment, early stopping, and model check-pointing ensures the best configurations are identified without overfitting. The results of these experiments are stored, and the optimal kernel and pooling sizes (k^* , p^*) are selected based on performance metrics such as accuracy, precision, recall, and F1 score. Step 3 further optimizes the model by experimenting with batch sizes ranging from 2 to 256 (2^N : $N = 1$ to 8). The batch size that yields the highest performance is then chosen as b^* .

The algorithm emphasizes robustness by incorporating cross-dataset evaluation in Step 4, where the optimized configuration (k^* , p^* , b^*) is validated on datasets [D1, D2, ..., D5] to assess its generalization to unseen data. This step ensures that the model is not overly tuned to the training dataset and can effectively classify images across diverse datasets. Finally in Step 5, the algorithm's emphasis on cross-dataset evaluation helps to ensure that the final model can effectively classify brain tumors across different MRI datasets, providing a reliable solution for MRI Image classification.

Input: MRI image datasets $[D_0, D_1, \dots, D_5]$

Output: Optimal CNN model configuration (k^*, p^*, b^*) for best classification performance

1: Step1: Initialize Parameters

```

2:  $D \leftarrow [D_0, D_1, \dots, D_5]$  // MRI Brain tumor Image dataset
3:  $K \leftarrow [(1, 1), (2, 2), \dots, (12, 12)]$  // Convolutional kernels
4:  $P \leftarrow [(2, 2), (3, 3), \dots, (9, 9)]$  // Max pooling sizes
5:  $L \leftarrow [32, 64, 128]$  //number for kernels in Convolutional layers 1,2,3
6:  $b \leftarrow 32$  // Batch size
7:  $lr \leftarrow 0.001$  // Initial learning rate for adaptive adjustment
8:  $min\_lr \leftarrow 1e-6$  //Minimum learning rate
9:  $ep \leftarrow 64$  // Number of epochs
10:  $fr \leftarrow 0.5$  // Learning rate adjustment factor
11:  $d \leftarrow 512$  // Number of neurons in the dense layer
12:  $dp \leftarrow 0.35$  // Dropout rate
13:  $o \leftarrow Adam$  // Optimizer
14:  $loss \leftarrow categorical-cross-entropy$  // Loss function
15:  $pt \leftarrow 6$  //Early stopping patience value

```

16: Step 2: Kernel and Pooling Size Optimization

```

17: for  $k \in K$  do
18:   for  $p \in P$  do
19:     try:
20:       Build model  $M_{kp}$  with the following layers:
21:       Convolutional layers with kernel size  $k \times k$ 
22:       Max pooling layers with pooling size  $p \times p$ 
23:       Dense layers with  $d$  neurons and  $dp$  dropout rate
24:       Output layer with softmax activation for the defined number of classes
25:       Define Callbacks:
26:       Reduce learning rate when validation loss plateaus from  $lr$  to  $min\_lr$ 
27:       Save the model in the checkpoint with the best validation accuracy
28:       Stop training if no improvement after  $pt$  epochs
29:       Compile the model  $M_{kp}$  using:
30:       Optimizer  $o$  and  $loss$  function
31:       Train  $M_{kp}$  on the dataset  $D_0$  using batch size  $b$ :
32:       for each epoch do
33:         Fit the model using train_generator and test_generator
34:         Handle Callbacks for  $lr$  adjustment, checkpoint, and early stopping
35:       end for
36:       Evaluate  $M_{kp}$  with performance metrics
37:       Calculate test loss and accuracy
38:       Generate classification Report with Precision, recall, F1-score
39:       Store results  $R[k, p]$  in a results table and save the model
40:     catch:
41:       Print "Model  $M_{kp}$  failed due to error"
42:       Continue to the next iteration
43:   end for
44: end for

```

45: Identify the best combination (k^*, p^*) based on performance from $R[k, p]$

46: Step 3: Batch Size Optimization

```

47: for  $b \in [2, 4, 8, 16, 32, 64, 128, 256]$  do
48:   Train model  $M_{k^*p^*}$  on  $D_0$  with batch size  $b$ 
49:   Evaluate  $M_{k^*p^*}$  and store the performance metrics in results table  $R[b]$ 
50: end for

```

51: Identify the best batch size b^* from $R[B]$

52: Step 4: Cross-Dataset Evaluation

```

52: for  $d \in D \setminus \{D_0\}$  do
53:   Evaluate the model  $M_{k^*p^*}$  with  $b^*$  on  $d$  using performance metrics
54: end for

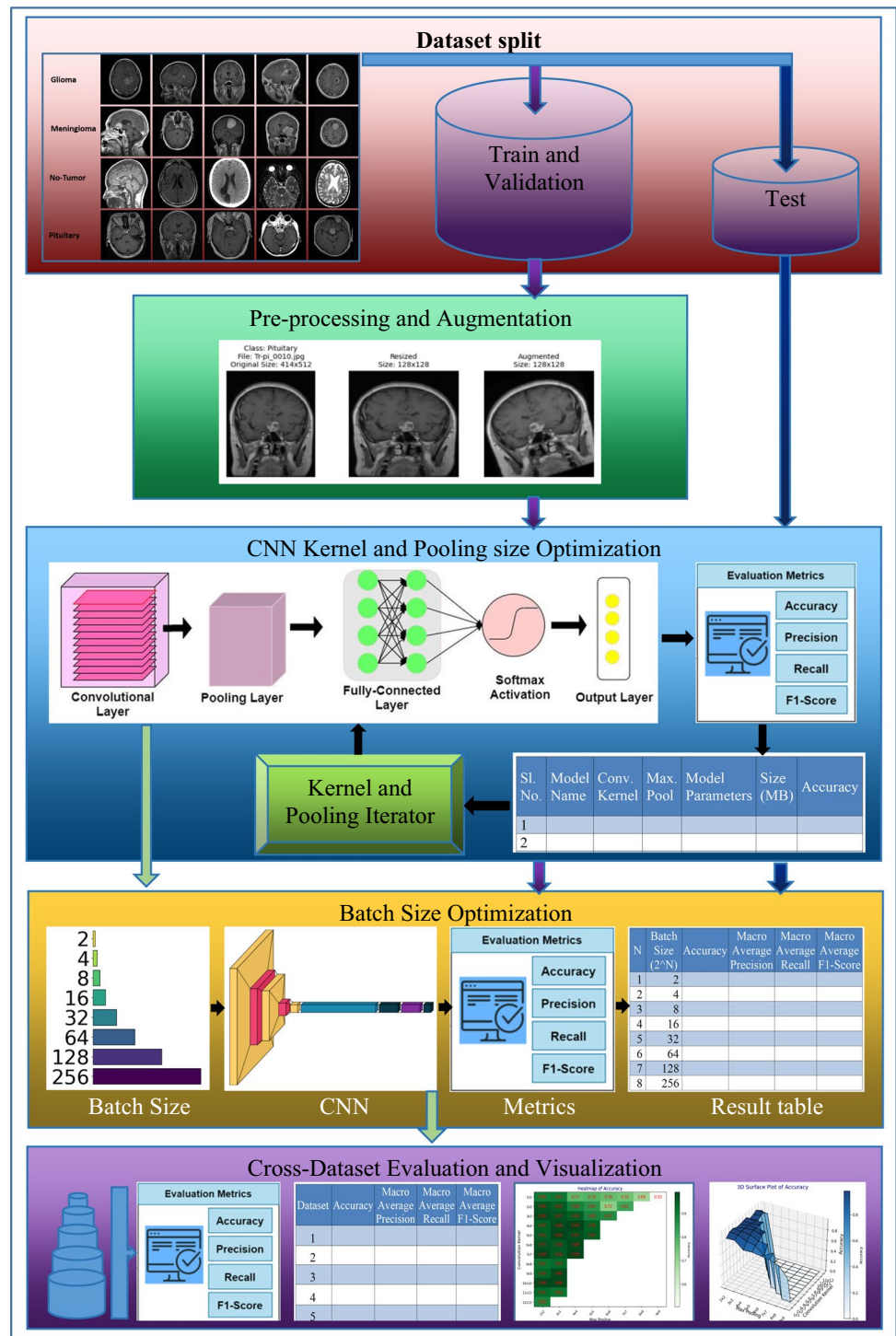
```

55: Step 5: Output Results

56: **return** the optimal configuration (k^*, p^*, b^*) and evaluation metrics

The proposed workflow of the MRI brain tumor classification approach as shown in Fig. 1 begins with splitting the dataset into training and testing subsets. The next step involves pre-processing and augmentation, which includes re-sizing images to a standard size, normalizing pixel values from 0 to 1, and applying techniques like rotation, brightness, shift, sheer, zoom, fill, and flip to increase the diversity of the training data. Following this, the algorithm optimizes kernel and pooling sizes to enhance feature extraction by experimenting with various configurations. The batch size is then optimized to determine the most effective training speed and model performance. The final step involves cross-dataset evaluation, where the best model is tested on different datasets to ensure generalization. The approach concludes with

Fig. 1 The workflow diagram of the proposed MRI classification approach



visualization, displaying performance metrics such as accuracy, precision, recall, F1-score, and confusion matrices to assess the model's overall effectiveness.

Data normalization and augmentation are essential for enhancing model performance and generalization. MRI images are resized to (150,150) to maintain consistency across datasets while preserving key tumor features. Image normalization is applied using rescaling (1./255) to normalize pixel values between 0 and 1, enhancing training stability and convergence. To improve model generalization, we incorporate rotation (10°), brightness adjustments (0.85–1.15), shear (12.5), minimal width/height shifts (0.002), and horizontal flipping, ensuring that MRI spatial integrity is maintained.

3.3 Model architecture

The proposed lightweight CNN architecture consists of several Convolutional layers followed by pooling layers, with dropout layers to prevent overfitting. The architecture was designed to balance depth and complexity, resulting in a lightweight model that performs well on the MRI brain tumor classification task. The model is trained using the Adam optimizer, with learning rate adjustments implemented through a custom learning rate scheduler. This model comprises the combinations of five main layers: Convolutional layers, max-pooling layers, a flattened layer, dense layers, and a dropout layer. The model starts by processing input images, gradually extracting higher-level features, and ends with a classification layer that predicts the tumor type.

Algorithm 1 suggests 96 CNN architectures but only 42 CNN architectures run successfully. These successful CNN models are categorized into three groups based on the number of trainable parameters and size, they are larger, moderate, and smaller models. Larger models, with over 10 million parameters, include 16 models including M12, M13, M22, M23, M32, M33, M42, M43, M44, M45, M52, M62, M72, M82, M48, and M49. Moderate models, with parameters ranging between 2 and 10 million, comprise 6 models, including M14, M15, M16, M17, M18, M19, M24, M34, and M35. Smaller models, with less than 2 million parameters, include 20 models such as M16, M17, M18, M25, M26, M27, M35, M45, M54, M74, M53, M55, M63, M64, M55, M52, and M51. This classification provides a clear distinction between models based on their size, helping in understanding their computational complexity and suitability for various applications.

The moderate CNN model M44 is explained for better understanding and is shown in Fig. 2, it has five layers of trainable parameters out of nine layers with computational tasks. In pre-processing all classes of MRI brain tumor images are re-sized to 150×150 , which is the input to the first layer of the model. The first layer is a Convolutional layer (Conv2D) with 32 filters of size (4, 4), which is processed with the Convolutional kernels to generate feature maps. Each filter applies convolution to the image, extracting patterns such as edges, textures, and shapes. The number of parameters for this layer is calculated by the equation, $\text{Parameters} = [((\text{Kernel Width} \times \text{Kernel Height} \times \text{Input Channels}) + 1(\text{for bias})) \times \text{Number of Filters}]$. Based on the kernel size (4 × 4) and input channels 3 with 32 filters, the calculation $[((4 \times 4 \times 3) + 1) \times 32]$ results in 1,568 parameters. Following this, a max pooling layer (MaxPooling2D) is used with a pool size of (4, 4) to reduce the spatial dimensions of the output from the Convolutional layer, from 147×147 to 36×36 . Max pooling does not have any parameters in training, it just reduces the dimensions.

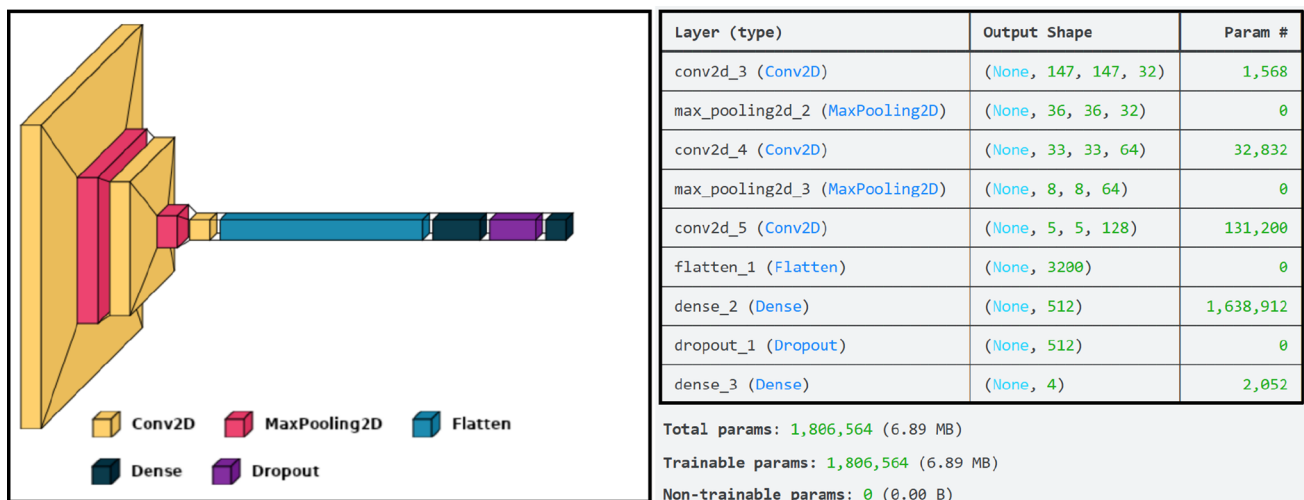


Fig. 2 CNN M44 architecture

The second Convolutional layer (Conv2D) is similar in structure to the first but has 64 filters and operates on the output of the previous max pooling layer. The number of parameters for this layer is calculated to be 32,832. Again, this is followed by another max pooling layer (MaxPooling2D) with the same pool size, which reduces the spatial dimensions from 33×33 to 8×8 . This max pooling layer does not introduce any additional parameters. The third Convolutional layer (Conv2D) has 128 filters, each of size (4, 4), and works on the output from the second max pooling layer. This layer has 131,200 parameters. The output of this layer is then flattened, which converts the 3D feature map ($5 \times 5 \times 128$) into a 1D vector with 3200 units. The flattened layer does not introduce any parameters.

Next, the model includes a dense layer with 512 neurons, which performs a fully connected operation. This layer has 1,638,912 parameters, as it connects every unit in the flattened layer to every unit in the dense layer. A dropout layer with a rate of 0.35 follows the dense layer to reduce overfitting during training. The dropout layer does not have any learnable parameters. Finally, the output layer is another dense layer with 4 units, corresponding to the 4 tumor types the model is classifying. The number of parameters for this layer is 2,052.

In total, the model has **1,806,564 parameters**. This architecture efficiently combines Convolutional and fully connected layers, with dropout applied to regularize the model. The total number of parameters indicates a relatively complex model, capable of learning rich features from the input images while being well-suited for classification tasks like brain tumor detection.

3.4 Performance metrics

All CNN performance can be evaluated using diverse evaluation metrics Accuracy, Precision, Recall, and F1 score explained briefly with the following 7 equations.

Accuracy measures the overall performance of the model by calculating the ratio of correctly predicted instances to the total number of instances. It reflects how often the model is correct across all classes.

$$\text{Accuracy} = \frac{\sum_{i=1}^N TP_i}{\text{Total Instances}} \quad (1)$$

Macro-Averaged Precision is the average of the precision scores for each class. It treats each class equally regardless of its size, providing a measure of performance for each class independently.

$$\text{Macro Average Precision} = \frac{1}{N} \sum_{i=1}^N \frac{TP_i}{TP_i + FP_i} \quad (2)$$

Macro-Averaged Recall is the average of the recall scores for each class. It measures how well each class is detected, treating each class equally regardless of its size.

$$\text{Macro Average Recall} = \frac{1}{N} \sum_{i=1}^N \frac{TP_i}{TP_i + FN_i} \quad (3)$$

Macro-Averaged F1 Score calculates the average F1 score for each class, treating each class equally. It balances precision and recall for each class and provides an average measure across all classes.

$$\text{Macro Average F1 Score} = \frac{1}{N} \sum_{i=1}^N 2 \times \frac{\text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i} \quad (4)$$

Weighted-Averaged Precision accounts for the number of instances in each class. It gives more importance to classes with more instances, reflecting the performance of the model in relation to the distribution of classes.

$$\text{Weighted Average Precision} = \frac{\sum_{i=1}^N \left(\frac{TP_i}{TP_i + FP_i} \times \text{Support}_i \right)}{\sum_{i=1}^N \text{Support}_i} \quad (5)$$

Weighted-Averaged Recall considers the number of instances in each class, giving more weight to classes with more instances. It measures the model's ability to detect each class, taking into account class imbalance.

$$\text{Weighted Average Recall} = \frac{\sum_{i=1}^N \left(\frac{TP_i}{TP_i + FP_i} \times \text{Support}_i \right)}{\sum_{i=1}^N \text{Support}_i} \quad (6)$$

Weighted-Averaged F1 Score combines precision and recall for each class, considering the class size. It provides a balanced measure of performance by accounting for the number of instances in each class.

$$\text{Weighted Average F1 Score} = \frac{\sum_{i=1}^N \left(2 \times \frac{\text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i} \times \text{Support}_i \right)}{\sum_{i=1}^N \text{Support}_i} \quad (7)$$

N = Number of classes. TP_i = True positives for class i. TN_i = True negative for class i. FP_i = False positives for class i. FN_i = False negatives for class i. Total instances = $\text{sum}(TP_i, TN_i, FP_i, FN_i)$. Precision_i = Precision for class i. Recall_i = Recall for class i. Support_i = Number of true instances for class i.

The performance of all CNN models is evaluated using key metrics: Accuracy, Macro-Averaged Precision, Recall, F1 Score, and their weighted counterparts. Macro metrics treat each class equally, while weighted metrics account for class imbalance by giving more importance to larger classes. These metrics assess model effectiveness in correctly predicting and detecting instances across different classes, with formulas based on true positives, false positives, true negatives, and false negatives.

4 Results and discussion

In this section, the 42 custom-built CNN architectures based on the combinations of Convolutional kernel sizes and max pooling sizes with their performance metrics were analyzed. The objective of these experiments was to assess the impact of these parameters on model performance, including accuracy, precision, recall, and F1-score, with a particular emphasis on their suitability for brain tumor classification in MRI images with different batch sizes. The main goal is to derive valuable insights that can guide to design of efficient and high-performing lightweight CNN models for medical image analysis [26, 27].

4.1 Convolutional kernel and max pooling size optimization

The results from the comparison of custom-built CNN models presented in Table 2 indicate that the selection of Convolutional kernel and max pooling sizes plays a significant role in model performance. The algorithm proposed 96 possible configurations based on 12 different kernel sizes ranging from (1×1) to (12×12) and 8 different pooling sizes ranging from (2×2) to (9×9) . However, due to memory and computational constraints, only 42 models were successfully executed. Among these, medium to larger kernel sizes, such as $(4, 4)$ and $(5, 5)$, combined with moderate pooling sizes like $(4, 4)$ and $(3, 3)$, consistently yielded superior performance in terms of accuracy, precision, recall, and F1-score. For instance, Model M44, with a $(4, 4)$ kernel and $(4, 4)$ pooling, achieved an accuracy of 99.01% along with high macro-average precision, recall, and F1-score values. In contrast, smaller kernel sizes, such as $(1, 1)$, or large pooling sizes, like $(8, 8)$ and $(9, 9)$, resulted in significantly lower accuracy, with models like M18 and M19 showing accuracy scores of 69.26% and 50.27%, respectively. This suggests that smaller kernels and large pooling sizes are less effective at capturing the detailed patterns inherent in MRI images.

Additionally, models with higher numbers of parameters, such as Model M12, which had over 89 million parameters, experienced execution failures due to excessive computational demands, while models with fewer parameters, like Model M18 (275,268 parameters), were faster but performed poorly. These findings underestimate the tradeoff between model complexity and computational resources, where models with larger kernels and moderate pooling sizes offered the best balance of performance and resource efficiency for brain tumor classification.

Another source of error observed in some models was the mismatch between kernel size, pooling size, and input dimensions. For example, Model M66, with a $(6, 6)$ kernel and $(6, 6)$ pooling on an input tensor of size $(3, 3)$, caused an invalid configuration, resulting in negative output dimensions. This issue could be resolved by reducing the kernel or pooling size. Furthermore, models such as M11, with a 1×1 kernel and 1×1 pooling, faced frequent kernel crashes during training due to the large memory requirement (5.49 GB) and the vast number of parameters (1,474,573,124). Similarly,

Table 2 CNN models performance comparison (convolutional kernel and max pooling)

Sl. No	Model Name	Conv. Kernel	Max. Pool	Model Parameters	Size (MB)	Accuracy
1	M12	(1, 1)	(2, 2)	89,731,908	342.30	0.8986
2	M13	(1, 1)	(3, 3)	16,790,340	64.05	0.9062
3	M14	(1, 1)	(4, 4)	5,321,540	20.30	0.7735
4	M15	(1, 1)	(5, 5)	2,372,420	9.05	0.7849
5	M16	(1, 1)	(6, 6)	1,061,700	4.05	0.7643
6	M17	(1, 1)	(7, 7)	602,948	2.30	0.7468
7	M18	(1, 1)	(8, 8)	275,268	1.05	0.6926
8	M19	(1, 1)	(9, 9)	78,660	0.30	0.5027
9	M22	(2, 2)	(2, 2)	80,325,732	306.42	0.9642
10	M23	(2, 2)	(3, 3)	14,789,732	56.42	0.9527
11	M24	(2, 2)	(4, 4)	4,238,436	16.17	0.8841
12	M25	(2, 2)	(5, 5)	1,092,708	4.17	0.8520
13	M26	(2, 2)	(6, 6)	306,276	1.17	0.7170
14	M27	(2, 2)	(7, 7)	109,668	0.42	0.8154
15	M32	(3, 3)	(2, 2)	75,855,428	289.37	0.9786
16	M33	(3, 3)	(3, 3)	11,171,396	42.62	0.9680
17	M34	(3, 3)	(4, 4)	2,455,108	9.37	0.9504
18	M35	(3, 3)	(5, 5)	685,636	2.62	0.8841
19	M36	(3, 3)	(6, 6)	161,348	0.62	0.8680
20	M42	(4, 4)	(2, 2)	67,277,028	256.64	0.9466
21	M43	(4, 4)	(3, 3)	9,605,348	36.64	0.9771
22	M44	(4, 4)	(4, 4)	1,806,564	6.89	0.9901
23	M45	(4, 4)	(5, 5)	430,308	1.64	0.9436
24	M52	(5, 5)	(2, 2)	59,243,588	226.00	0.9512
25	M53	(5, 5)	(3, 3)	6,814,788	26.00	0.9832
26	M54	(5, 5)	(4, 4)	1,309,764	5.00	0.9733
27	M55	(5, 5)	(5, 5)	326,724	1.25	0.9519
28	M62	(6, 6)	(2, 2)	51,755,108	197.43	0.9687
29	M63	(6, 6)	(3, 3)	5,683,300	21.68	0.9611
30	M64	(6, 6)	(4, 4)	637,028	2.43	0.9825
31	M72	(7, 7)	(2, 2)	48,284,996	184.19	0.9847
32	M73	(7, 7)	(3, 3)	4,703,556	17.94	0.9603
33	M74	(7, 7)	(4, 4)	574,788	2.19	0.9863
34	M82	(8, 8)	(2, 2)	41,624,292	158.78	0.9169
35	M48	(8, 8)	(3, 3)	3,023,588	11.53	0.9306
36	M49	(9, 9)	(2, 2)	35,508,548	135.45	0.9405
37	M50	(9, 9)	(3, 3)	2,478,404	9.45	0.9779
38	M51	(10, 10)	(2, 2)	29,937,764	114.20	0.9817
39	M52	(10, 10)	(3, 3)	1,626,212	6.20	0.9825
40	M53	(11, 11)	(2, 2)	27,467,844	104.78	0.9314
41	M54	(11, 11)	(3, 3)	1,515,588	5.78	0.9115
42	M55	(12, 12)	(2, 2)	22,724,836	86.69	0.8833
Sl. No	Macro Average Precision	Macro Average Recall	Macro Average F1-Score	Weighted Average Precision	Weighted Average Recall	Weighted Average F1-Score
1	0.9017	0.8908	0.8915	0.9036	0.8986	0.8966
2	0.9164	0.8983	0.9010	0.9154	0.9062	0.9048
3	0.8092	0.7563	0.7577	0.8067	0.7735	0.7663
4	0.7987	0.7677	0.7692	0.7948	0.7849	0.7758

Table 2 (continued)

Sl. No	Macro Average Precision	Macro Average Recall	Macro Average F1-Score	Weighted Average Precision	Weighted Average Recall	Weighted Average F1-Score
5	0.7679	0.7476	0.7443	0.7690	0.7643	0.7536
6	0.7404	0.7269	0.7188	0.7410	0.7468	0.7290
7	0.7117	0.6690	0.6652	0.7060	0.6926	0.6732
8	0.4954	0.4811	0.4715	0.5056	0.5027	0.4884
9	0.9637	0.9614	0.9624	0.9641	0.9642	0.9640
10	0.9559	0.9491	0.9519	0.9535	0.9527	0.9524
11	0.8890	0.8744	0.8774	0.8876	0.8841	0.8816
12	0.8641	0.8400	0.8441	0.8589	0.8520	0.8473
13	0.7760	0.6942	0.6994	0.7620	0.7170	0.7027
14	0.8288	0.8015	0.7984	0.8239	0.8154	0.8028
15	0.9796	0.9768	0.9780	0.9790	0.9786	0.9786
16	0.9701	0.9655	0.9673	0.9684	0.9680	0.9676
17	0.9549	0.9460	0.9482	0.9538	0.9504	0.9500
18	0.8909	0.8747	0.8775	0.8874	0.8841	0.8804
19	0.8710	0.8581	0.8590	0.8679	0.8680	0.8622
20	0.9473	0.9425	0.9432	0.9492	0.9466	0.9463
21	0.9761	0.9754	0.9757	0.9771	0.9771	0.9771
22	0.9906	0.9893	0.9899	0.9901	0.9901	0.9901
23	0.9466	0.9389	0.9409	0.9464	0.9436	0.9431
24	0.9528	0.9477	0.9494	0.9523	0.9512	0.9509
25	0.9836	0.9819	0.9826	0.9833	0.9832	0.9832
26	0.9732	0.9710	0.9717	0.9740	0.9733	0.9732
27	0.9528	0.9479	0.9500	0.9523	0.9519	0.9518
28	0.9686	0.9661	0.9669	0.9692	0.9687	0.9686
29	0.9641	0.9577	0.9596	0.9631	0.9611	0.9608
30	0.9826	0.9810	0.9817	0.9825	0.9825	0.9824
31	0.9849	0.9835	0.9842	0.9847	0.9847	0.9847
32	0.9617	0.9571	0.9590	0.9607	0.9603	0.9601
33	0.9861	0.9851	0.9855	0.9864	0.9863	0.9862
34	0.9238	0.9104	0.9118	0.9240	0.9169	0.9155
35	0.9343	0.9254	0.9286	0.9316	0.9306	0.9298
36	0.9415	0.9360	0.9366	0.9439	0.9405	0.9402
37	0.9777	0.9759	0.9765	0.9785	0.9779	0.9779
38	0.9811	0.9801	0.9803	0.9822	0.9817	0.9817
39	0.9817	0.9811	0.9812	0.9829	0.9825	0.9825
40	0.9335	0.9259	0.9280	0.9341	0.9314	0.9311
41	0.9167	0.9051	0.9063	0.9197	0.9115	0.9113
42	0.8826	0.8758	0.8772	0.8822	0.8833	0.8807

Bold values indicated achieves high accuracy

Model M21, despite having slightly fewer parameters, also faced memory limitations and kernel shutdowns. These examples highlight the importance of optimizing model complexity for efficient training on systems with limited memory. In conclusion, the combination of kernel and pooling sizes, along with the model's parameter count, significantly influences the performance and feasibility of CNN models for MRI brain tumor classification tasks. Moderate kernel and pooling sizes, balanced with a manageable number of parameters, resulted in the best-performing models.

An analysis of various convolutional kernel and max pooling size combinations reveals the substantial impact of pooling size on the number of learnable parameters in CNN models. As the max pooling size increases, the feature maps undergo more aggressive downsampling, which significantly reduces spatial dimensions and, consequently, the size of

subsequent layers (particularly the fully connected layers), leading to a drastic reduction in the total number of learnable parameters. There is no single optimal kernel and max pooling size combination for achieving the best accuracy; rather, specific combinations can yield better performance by effectively capturing essential features from the image.

For a convolutional kernel of (1×1) , when the max pooling size increases from (2×2) to (9×9) , the number of learnable parameters is significantly reduced from 89,731,908 (89.86%) to just 78,660 (50.27%), respectively. Similarly, for a convolutional kernel of (2×2) , increasing the max pooling size from (2×2) to (7×7) reduces the parameters from 80,325,732 (96.42%) to 109,668 (81.54%). For a convolutional kernel of (3×3) , increasing the pooling size from (2×2) to (6×6) reduces the parameters from 75,855,428 (97.86%) to 161,348 (86.80%). Likewise, with a pooling size increase from (2×2) to (4×4) for a convolutional kernel of (4×4) , the number of learnable parameters drops from 67,277,028 (94.66%) to 1,806,564 (99.01%).

These observations indicate that the size of max pooling plays a dominant role in determining the total number of learnable parameters in a CNN model. Increasing the max pooling size compresses the feature maps more aggressively, thus reducing the dimensions and, in turn, the parameter count in subsequent layers (especially dense layers). Therefore, selecting an appropriate combination of convolutional kernel size and max pooling size is crucial for designing an efficient CNN architecture. The combination of (4×4) kernel and (4×4) max pooling helps to optimize the receptive field to capture more meaningful features from the MRI input image, ultimately improving classification accuracy.

Table 3 presents a consolidated accuracy comparison of different Convolutional kernel sizes and max pooling combinations based on the results from Table 2. The table highlights the accuracy performance of various combinations, with Convolutional kernel sizes ranging from $(1, 1)$ to $(12, 12)$ and max pooling sizes from $(2, 2)$ to $(9, 9)$. The results indicate that smaller kernel sizes, such as $(3, 3)$ and $(4, 4)$, consistently achieve higher accuracy compared to larger kernel sizes like $(8, 8)$ and $(9, 9)$. Notably, the $(3, 3)$ kernel and $(3, 3)$ max pooling combination shows impressive results with an accuracy of 97.86%. As the kernel size increases, accuracy tends to increase with larger max pooling sizes but not always. So there is no standard pattern between kernel and max pooling to maximize accuracy. In Table 3, some combinations achieve high accuracy but some combinations are marked as NA, indicating that certain configurations were not evaluated due to memory constraints and non-fitting architectures. The optimal accuracy performance is observed at the $(4, 4)$ kernel and $(4, 4)$ max pooling combination, yielding the highest accuracy of 99.01% as highlighted in Table 2 and 3 in bold.

Figure 3 visually represents the accuracy values of Table 3, showing different combinations of Convolutional kernel and max pooling sizes with four decimal places. The heatmap uses the viridis colormap, transitioning from dark purple to bright yellow, where darker shades indicate lower values and lighter shades of yellow represent higher values. Each cell displays the exact accuracy, aiding in the quick identification of optimal kernels and pooling size combinations for improved model performance.

Figure 4 displays a 3D surface plot of accuracy values for different combinations of Convolutional kernel sizes and max pooling sizes. The plot uses the viridis colormap to represent accuracy, with the height of the surface corresponding to the accuracy value. This visualization helps to identify how the accuracy varies across different kernels and pooling size combinations.

4.2 Batch size optimization

Batch size optimization is crucial because it directly influences the training efficiency, model performance, and convergence behavior in deep learning models. A well-chosen batch size can lead to faster training times and improved model accuracy.

Table 4 summarizes the performance metrics of model M44 across varying batch sizes, ranging from 2 to 256 ($2N^{\wedge}NN$, where $N = 1N = 1N = 1$ to 8), highlighting the influence of batch size on model performance. At a batch size of 32, the model achieves an accuracy of 99.0%, which improves to the highest accuracy of 99.5% at a batch size of 64, along with a Macro Average F1-Score and Weighted Average F1-Score of 99.5%, marking it as the optimal configuration. That is, the CNN model M44 with Convolutional kernel is (4×4) , max pooling is (4×4) and batch size is 64 achieves high accuracy, as emphasized in Table 4 (bolded).

The second-best performance is observed at a batch size of 128, with slightly lower but still better results. A bell curve trend is evident, where performance initially improves with increasing batch size, peaks at 64, and gradually declines with larger batch sizes. Table 4 indicates that moderate batch sizes can have computational efficiency, which may lead to the best performance. The table highlights the importance of selecting the right batch size like 64 to achieve the best results.

Table 3 Accuracy table for convolutional kernel and max pooling combinations

Conv. Kernel	Max Pooling							
	(2, 2)	(3, 3)	(4, 4)	(5, 5)	(6, 6)	(7, 7)	(8, 8)	(9, 9)
(1, 1)	0.8986	0.9062	0.7735	0.7849	0.7643	0.7468	0.6926	0.5027
(2, 2)	0.9642	0.9527	0.8841	0.8520	0.7170	0.8154	NA	NA
(3, 3)	0.9786	0.9680	0.9504	0.8841	0.8680	NA	NA	NA
(4, 4)	0.9466	0.9771	0.9901	0.9436	NA	NA	NA	NA
(5, 5)	0.9512	0.9832	0.9733	0.9519	NA	NA	NA	NA
(6, 6)	0.9687	0.9611	0.9825	NA	NA	NA	NA	NA
(7, 7)	0.9847	0.9603	0.9863	NA	NA	NA	NA	NA
(8, 8)	0.9169	0.9306	NA	NA	NA	NA	NA	NA
(9, 9)	0.9405	0.9779	NA	NA	NA	NA	NA	NA
(10, 10)	0.9817	0.9825	NA	NA	NA	NA	NA	NA
(11, 11)	0.9314	0.9115	NA	NA	NA	NA	NA	NA
(12, 12)	0.8833	NA	NA	NA	NA	NA	NA	NA

Bold values indicates the achieves high accuracy

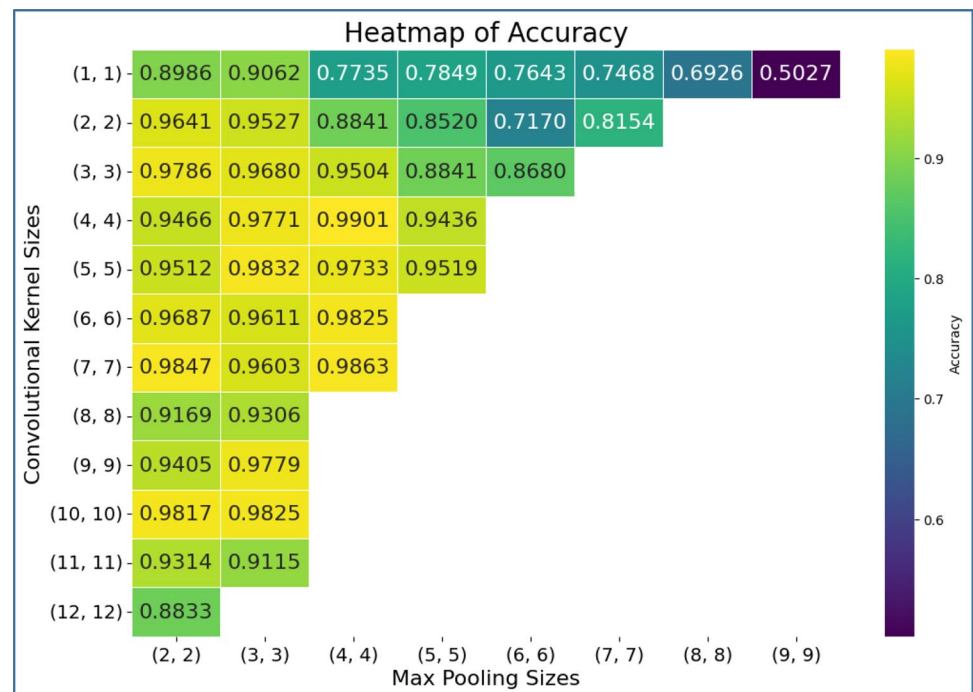
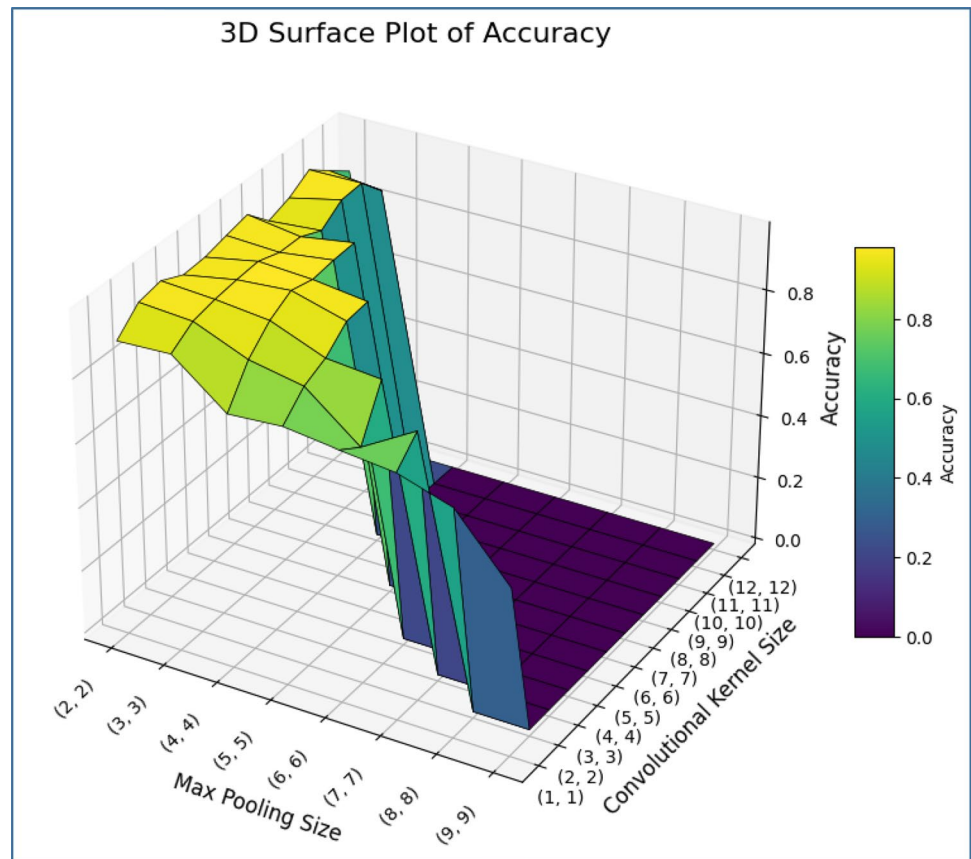
Fig. 3 Accuracy heatmap for convolutional kernel and max pooling combinations

Figure 5 provides a detailed evaluation of Model M44 (batch size = 64) using its accuracy graph, loss graph, ROC curve, and confusion matrix, highlighting its outstanding performance in MRI brain tumor classification. The accuracy graph demonstrates a consistently increasing trend in training and validation accuracy, culminating in an exceptional validation accuracy of 99.54%, which reflects effective learning with minimal overfitting. The loss graph further emphasizes the model's robustness, showing a steady decline and convergence to minimal training and validation loss values. The ROC curve achieves a perfect AUC of 1 across all four classes (glioma, meningioma, no tumor, and pituitary), indicating superior discriminatory ability and precise class separation. The confusion matrix showcases the model's accuracy, with the majority of predictions correctly matching their actual labels; for example, all images in pituitary tumor class and no tumor class are classified correctly, 303 out of 305 meningioma image samples and 297 out of 300 glioma samples were accurately classified, with similar high performance for the other classes. These comprehensive metrics collectively underscore the reliability, robustness, and practical applicability of Model M44 for real-world MRI brain tumor classification tasks.

Figure 6 visualizes the proposed model's classification performance by displaying four random samples of MRI images from four classes: glioma, meningioma, no tumor, and pituitary. For every input image, the true class and the model's

Fig. 4 3D surface accuracy plot**Table 4** Performance metrics of model M44 for the batch size [2, 4, 8, 16, 32, 64, 128, 256]

N	Batch Size (2 ^N)	Accuracy	Macro Average Precision	Macro Average Recall	Macro Average F1-Score	Weighted Average Precision	Weighted Average Recall	Weighted Average F1-Score
1	2	0.8017	0.7976	0.7866	0.7855	0.7960	0.8017	0.7917
2	4	0.8879	0.8870	0.8799	0.8815	0.8865	0.8879	0.8852
3	8	0.9710	0.9714	0.9690	0.9699	0.9710	0.9710	0.9707
4	16	0.9748	0.9750	0.9728	0.9738	0.9748	0.9748	0.9747
5	32	0.9901	0.9906	0.9893	0.9899	0.9901	0.9901	0.9901
6	64	0.9954	0.9950	0.9950	0.9950	0.9954	0.9954	0.9954
7	128	0.9947	0.9947	0.9942	0.9944	0.9947	0.9947	0.9947
8	256	0.9825	0.9824	0.9811	0.9816	0.9825	0.9825	0.9824

Bold values indicates the achieves high accuracy

predicted class are annotated in the title, along with the prediction probability in parentheses. The layout provides a clear understanding of the model's accuracy by showcasing consistent matches between true and predicted classes for all samples, along with confidence probability, highlighting the reliability of the predictions. The given images will serve as input, while the corresponding predicted labels and probabilities represent the output.

4.3 Cross dataset evaluation

Cross-dataset evaluation is crucial for assessing a model's ability to generalize to unseen data, helping to prevent overfitting and ensuring reliable performance across different datasets. It provides a more robust estimate of model effectiveness in real-world scenarios by testing it on diverse data, revealing potential issues like dataset bias or

to achieve the best results.

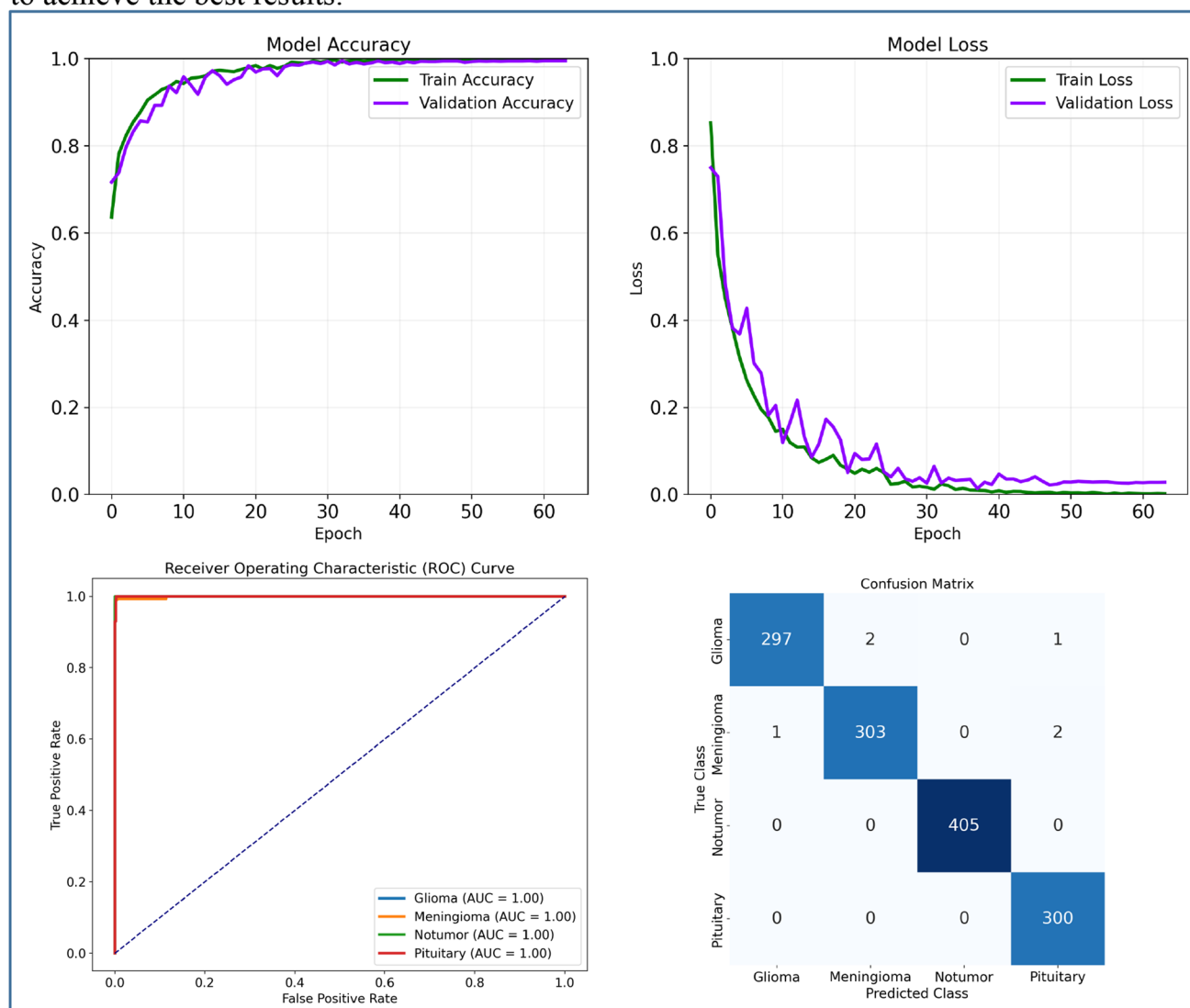


Fig. 5 Proposed model's accuracy graph, loss graph, ROC curve, and confusion matrix

quality problems. Additionally, this evaluation helps to improve performance metrics by reducing bias from a single dataset and aids in selecting the most reliable model configuration. Overall, cross-dataset evaluation ensures that the model is well-suited for practical deployment, offering insights into its real-world applicability and robustness.

Table 5 highlights the performance of model M44 (batch size = 64) across five MRI brain tumor datasets, each containing four tumor classes. The results demonstrate consistently high performance across all datasets, showcasing the model's strong classification capability. Dataset 1 achieved the highest accuracy of 99.95%, with corresponding F1-scores of 99.96% with macro average and 99.95% with weighted average, indicating high classification precision and recall. Similarly, Dataset 2 performed at an impressive level, with an accuracy of 99.92% and F1-scores of 99.93% with a macro average and 99.92% with a weighted average, slightly lower than Dataset 1 but still indicative of excellent performance. Dataset 3 reported an accuracy of 99.87%, with macro and weighted average F1-scores of 99.86%, reflecting robust performance despite minor variations in data characteristics.

Both Dataset 4 and Dataset 5 displayed similar results, with accuracy of 99.93% and F1-scores exceeding 99.93%. The macro and weighted average F1-scores were 99.93% and 99.93%, respectively, for both datasets, indicating consistent performance across diverse data sources. These results highlight the model's ability to generalize effectively

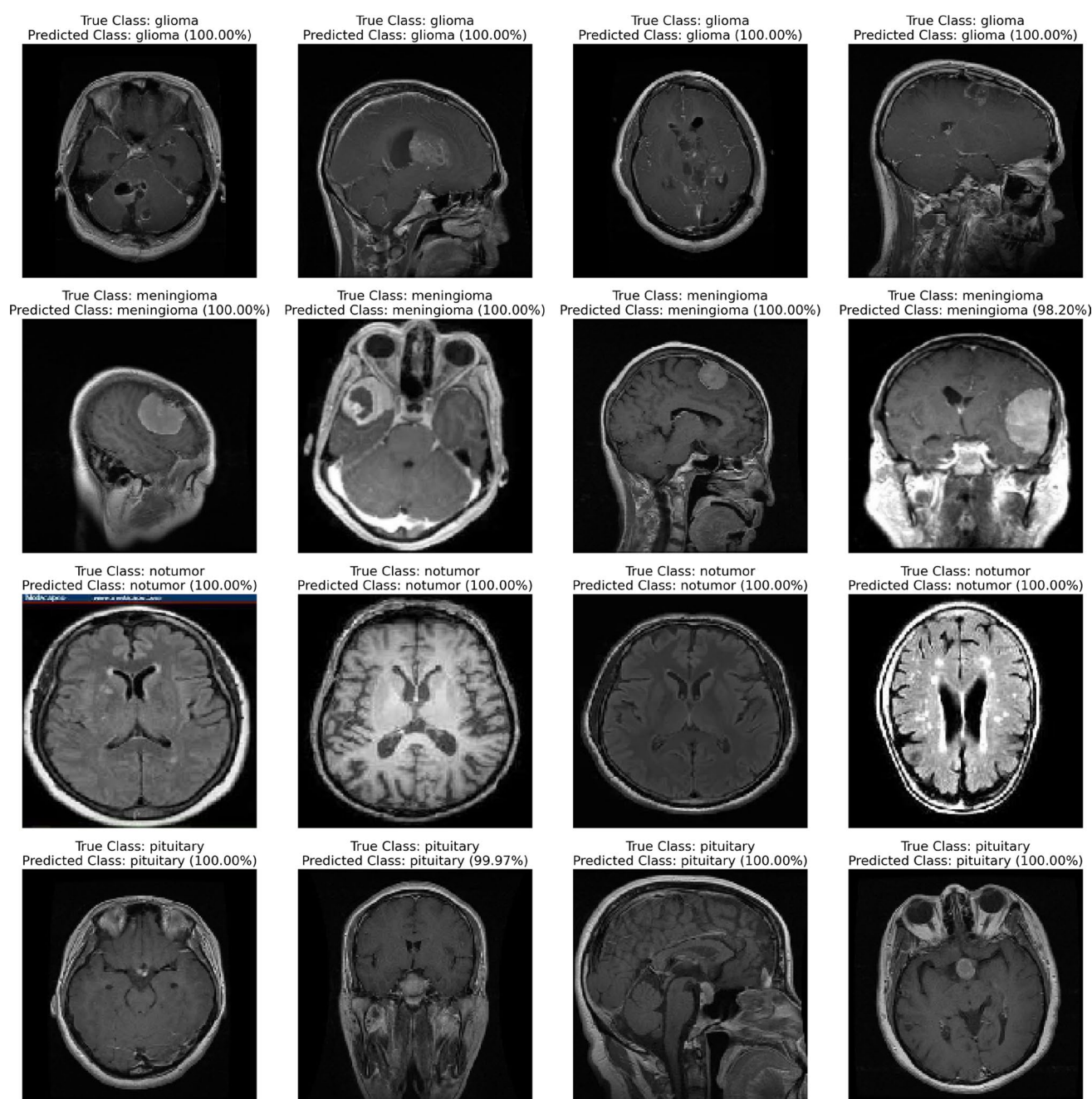


Fig. 6 Prediction of MRI images

Table 5 Performance metrics of cross-validation datasets (Model=M44 and batch size=64)

Dataset	Images	Accuracy	Macro Average Precision	Macro Average Recall	Macro Average F1-Score	Weighted Average Precision	Weighted Average Recall	Weighted Average F1-Score
1	2175	0.9995	0.9995	0.9996	0.9996	0.9995	0.9995	0.9995
2	5099	0.9992	0.9993	0.9992	0.9993	0.9992	0.9992	0.9992
3	5229	0.9987	0.9987	0.9986	0.9986	0.9987	0.9987	0.9987
4	5708	0.9993	0.9993	0.9993	0.9993	0.9993	0.9993	0.9993
5	5712	0.9993	0.9993	0.9993	0.9993	0.9993	0.9993	0.9993

across different datasets with minimal variation in performance. Overall, M44 exhibits exceptional reliability and robustness, with accuracies and F1-scores consistently exceeding 99.87% across all datasets. This consistent high performance demonstrates its suitability for real-world MRI brain tumor classification tasks, where diverse and heterogeneous data sources may be encountered.

4.4 Comparative analysis

The performance metrics of various CNN models in terms of size, parameter count, depth, and classification performance across multiple metrics, including accuracy, macro average, weighted average precision, recall, and F1 scores are tabulated in Table 6. Among the models, the Proposed Model stands out with the smallest size with 6.89 MB and 1.8 million parameters, achieving the highest accuracy of 99.54%, macro average F1-scores of 99.50%, and weighted average F1-scores of 99.54%, demonstrating superior efficiency and effectiveness. Comparatively, models like Xception and ResNet101V2 also show high accuracy of 96.95% and 97.41%, respectively, but with significantly larger sizes and parameter counts. Notably, lightweight models such as MobileNet perform well with an accuracy of 97.18%. Whereas deeper models like VGG16, VGG19, DenseNet201, NASNetLarge, and InceptionResNetV2 demonstrate strong performance with accuracies of 88.48%, 87.41%, 95.88%, 94.96%, and 94.81%, respectively. EfficientNetB0 attains an impressive 97.18% accuracy, placing it on par with MobileNet and outperforming several deeper architectures like DenseNet201. Overall, the Proposed Model with a Convolutional kernel size of (4×4) and max pooling (4×4) with batch size 64 demonstrates exceptional performance and efficiency, making it highly suitable for real-world applications with resource constraints.

Table 7 presents a comparison of various deep learning models used for brain tumor classification, including their key parameters, model sizes, and performance metrics with 4 decimal places. The models span a variety of architectures, such as ResNet, VGG, DenseNet, EfficientNet, and even a hybrid CNN-RNN approach, along with the proposed model. The performance is evaluated based on accuracy, macro average precision, recall, and F1-score, as well as weighted average precision, recall, and F1-score, providing a comprehensive view of each model's effectiveness. The proposed model demonstrates the highest accuracy of 99.54%, outperforming the others in all performance metrics, showcasing its potential for efficient brain tumor classification.

4.5 Ablation analysis

In this ablation analysis, we evaluated the impact of various hyperparameters and training strategies on the performance of our lightweight CNN model for brain tumor classification. Key factors considered includes kernel sizes, pooling strategies, batch sizes, and advanced optimization techniques such as adaptive learning rates, dropout, and early stopping. By systematically testing different configurations, we aimed to identify the optimal settings that would enhance the model's accuracy, F1-score, and overall generalization with cross-dataset evaluation.

Popular built-in Convolutional Neural Networks (CNNs) like VGG, ResNet, DenseNet, MobileNet, NASNet, Inception, Xception, EfficientNet, and SqueezeNet commonly use (3×3) kernels for Convolutional layers, although some models, such as Inception and Xception, utilize larger kernels like (5×5) and (7×7) . The pooling layers typically employ (2×2) max pooling. These kernel and pooling sizes balance computational efficiency and model performance, enabling the networks to capture important spatial features while managing complexity, which is essential for image classification. The combination of kernel and pooling sizes is crucial for striking a balance between computational efficiency and model performance, ensuring effective feature extraction while maintaining manageable model complexity.

The analysis of model parameters and size demonstrates how kernel and pooling configurations impact the model's memory and computational demands. Larger kernels (e.g., (12×12) or (11×11)) paired with smaller pooling sizes increase the number of parameters and result in larger models, while smaller kernels (e.g., (1×1)) with larger pooling sizes reduce the parameter count and memory usage, though they may affect the model's ability to extract detailed features. This highlights a trade-off between reducing computational costs and maintaining performance, where smaller kernels and pooling may lead to more computationally expensive models, whereas larger kernels reduce complexity but might limit feature extraction capabilities.

One of the primary focuses of this study was the choice of kernel sizes and pooling strategies. Usually. We tested combinations of kernel sizes ranging from $(1,1)$ to $(12,12)$ and pooling sizes from $(2,2)$ to $(9,9)$. The results revealed that a kernel size of $(3,3)$ and a pooling size of $(2,2)$ provided the best balance between efficient feature extraction

Table 6 Performance metrics of CNN models

Model	Parameters (Millions)	Size (MB)	Depth	Accuracy	Macro Avg Precision	Macro Avg Recall	Macro Avg F1-Score	Weighted Avg Precision	Weighted Avg Recall	Weighted Avg F1-Score
Xception [1, 16, 27, 28]	22.9	88	81	0.9695	0.9679	0.9669	0.9671	0.9699	0.9695	0.9694
VGG16 [15, 29, 30, 22]	138.4	528	16	0.8848	0.8860	0.8784	0.8801	0.8890	0.8848	0.8849
VGG19 [15, 29, 30]	143.7	549	19	0.8741	0.8715	0.8669	0.8684	0.8740	0.8741	0.8734
ResNet50 [23, 28, 31]	25.6	98	107	0.7429	0.7275	0.7327	0.7249	0.7334	0.7429	0.7333
ResNet50V2 [14, 28, 31]	25.6	98	103	0.9542	0.9543	0.9504	0.9505	0.9572	0.9542	0.9540
ResNet101 [16, 23, 32]	44.7	171	209	0.7315	0.7178	0.7144	0.7070	0.7209	0.7315	0.7171
ResNet101V2 [14, 31]	44.7	171	205	0.9741	0.9725	0.9719	0.9722	0.9740	0.9741	0.9740
ResNet152 [1, 24, 32, 22]	60.4	232	311	0.7605	0.7505	0.7456	0.7420	0.7529	0.7605	0.7506
ResNet152V2 [14, 24]	60.4	232	307	0.9428	0.9431	0.9397	0.9402	0.9459	0.9428	0.9432
InceptionV3 [2, 20, 30]	23.9	92	189	0.9329	0.9294	0.9277	0.9282	0.9332	0.9329	0.9327
InceptionResNetV2 [33, 23]	55.9	215	449	0.9481	0.9457	0.9450	0.9444	0.9496	0.9481	0.9480
MobileNet [2, 20, 34]	4.3	16	55	0.9718	0.9700	0.9694	0.9695	0.9721	0.9718	0.9717
MobileNetV2 [2, 20, 32]	3.5	14	105	0.9367	0.9345	0.9337	0.9331	0.9385	0.9367	0.9366
DenseNet121 [21, 24, 25]	8.1	33	242	0.9466	0.9436	0.9431	0.9424	0.9473	0.9466	0.9461
DenseNet169 [21, 35, 32]	14.3	57	338	0.9603	0.9592	0.9576	0.9579	0.9607	0.9603	0.9600
DenseNet201 [21, 24, 35]	20.2	80	402	0.9588	0.9564	0.9555	0.9558	0.9584	0.9588	0.9585
NASNetMobile [5, 36, 34]	5.3	23	389	0.9260	0.9215	0.9203	0.9201	0.9253	0.9260	0.9249
NASNetLarge [5, 36, 34]	88.9	343	533	0.9497	0.9494	0.9456	0.9463	0.9514	0.9497	0.9494
EfficientNetB0 [18, 26, 37]	5.3	29	132	0.9718	0.9711	0.9695	0.9698	0.9727	0.9718	0.9717
EfficientNetB1 [18, 26]	7.9	31	186	0.3089	0.0773	0.2500	0.1181	0.0955	0.3089	0.1459
EfficientNetB2 [27, 37]	9.2	36	186	0.3089	0.0772	0.2500	0.1180	0.0954	0.3089	0.1458
EfficientNetB3 [28, 22]	12.3	48	210	0.3082	0.0771	0.2494	0.1178	0.0953	0.3082	0.1455
EfficientNetB4 [28, 22]	19.5	75	258	0.2998	0.1015	0.2437	0.1236	0.1176	0.2998	0.1505
EfficientNetB5 [8, 25, 28]	30.6	118	312	0.2891	0.5150	0.2998	0.1816	0.5411	0.2891	0.1903
EfficientNetB6 [8, 38, 22]	43.3	166	360	0.3089	0.0772	0.2500	0.1180	0.0954	0.3089	0.1458
EfficientNetB7 [8, 38, 22]	66.7	256	438	0.3082	0.0771	0.2494	0.1178	0.0953	0.3082	0.1455
EfficientNetV2B0 [19, 37]	7.2	29	> 100	0.4294	0.2149	0.4000	0.2781	0.2360	0.4294	0.3030
EfficientNetV2B1 [19, 37]	8.2	34	> 120	0.9207	0.9364	0.9145	0.9203	0.9294	0.9207	0.9197
EfficientNetV2B2 [19, 39]	10.2	42	> 130	0.3089	0.0772	0.2500	0.1180	0.0954	0.3089	0.1458
EfficientNetV2B3 [19, 39]	14.5	59	> 150	0.3089	0.0772	0.2500	0.1180	0.0954	0.3089	0.1458
EfficientNetV2S [13, 40]	21.6	88	> 200	0.9474	0.9483	0.9429	0.9434	0.9500	0.9474	0.9467
EfficientNetV2M [13, 40]	54.4	220	> 250	0.3082	0.0771	0.2494	0.1178	0.0953	0.3082	0.1455
EfficientNetV2L [13, 40]	119.0	479	> 300	0.2235	0.0801	0.2388	0.0949	0.0819	0.2235	0.0897
Proposed Model	1.8	7	9	0.9954	0.9950	0.9950	0.9950	0.9954	0.9954	0.9954

Table 7 Performance metrics comparison with other related work

First author	Parameters (Millions)	Size (MB)	Accuracy	Macro average precision	Macro average recall	Macro average F1-score	Weighted average precision	Weighted average recall	Weighted average F1-score
Hossain et al. [41]	22.9	88	0.9695	0.9679	0.9669	0.9671	0.9699	0.9695	0.9694
Kumar et al. [25]	25.6	98	0.9542	0.9543	0.9504	0.9505	0.9572	0.9542	0.9540
Swati et al. [42]	4.3	16	0.9718	0.9699	0.9694	0.9695	0.9721	0.9718	0.9717
Ghassemi et al. [37]	23.9	92	0.9329	0.9294	0.9277	0.9282	0.9332	0.9329	0.9327
Liu et al. [38]	44.7	171	0.9741	0.9725	0.9719	0.9722	0.9740	0.9741	0.9740
Alanazi et al. [43]	20.2	80	0.9588	0.9564	0.9555	0.9558	0.9584	0.9588	0.9585
Alturki et al. [30]	10.2	42	0.9456	0.9471	0.9443	0.9452	0.9472	0.9456	0.9452
Hammad et al. [32]	5.3	23	0.9260	0.9215	0.9203	0.9207	0.9253	0.9260	0.9249
Tandel et al. [34]	60.4	232	0.9428	0.9431	0.9401	0.9405	0.9458	0.9429	0.9432
Sekhar et al. [28]	138.4	528	0.8848	0.8860	0.8784	0.8801	0.8890	0.8848	0.8849
Sasidhar et al. [39]	143.7	549	0.8741	0.8715	0.8669	0.8684	0.8740	0.8741	0.8734
Zhang et al. [31]	32.1	140	0.9790	0.9785	0.9770	0.9777	0.9795	0.9790	0.9788
Chen et al. [40]	50.3	180	0.9820	0.9810	0.9790	0.9800	0.9830	0.9820	0.9818
Patel et al. [22]	60.4	250	0.9870	0.9860	0.9850	0.9855	0.9880	0.9870	0.9868
Proposed work	1.8	7	0.9954	0.9950	0.9950	0.9950	0.9954	0.9954	0.9954

and computational cost, yielding high accuracy and F1-scores. These settings proved effective in capturing key tumor-related features while maintaining computational efficiency, making them optimal for this classification task.

Model M44, configured with a (4, 4) kernel and (4, 4) pooling size, has a total of 1,806,564 parameters and a memory size of 6.89 MB. Despite having a relatively moderate number of parameters, this model achieves an impressive accuracy of 99.01%. The choice of a (4, 4) kernel and pooling layer strikes a balance between model complexity and performance, allowing it to efficiently capture spatial features while keeping the computational cost relatively low. This configuration is well-suited for tasks that demand both high accuracy and a manageable model size. Larger kernels reduce convolution operations, lowering learnable parameters and computational complexity while maintaining accuracy.

Another crucial hyperparameter analyzed was batch size. We tested batch sizes ranging from 2 to 256 to understand their impact on training stability and model performance. Smaller batch sizes (e.g., 2, 4) led to lower performance, while larger batch sizes improved accuracy and F1-scores up to a point. Batch size 32 showed a significant improvement, with accuracy reaching 99.01% and F1-score rising to 98.99%. Performance peaked at batch size 64, with an accuracy of 99.54% and a weighted F1-score of 99.54%. However, further increases in batch size (e.g., 128 and 256) resulted in diminishing returns, highlighting the importance of choosing an optimal batch size for balancing computational efficiency and generalization. Based on these findings, batch size 64 was determined to be the best choice for maximizing model performance while minimizing over-fitting and computational burden.

The combination of optimal kernel and pooling sizes, along with appropriate batch size, significantly enhances the performance of the CNN model for brain tumor classification. The use of advanced techniques such as adaptive learning rates, dropout, and early stopping further contributed to the model's robustness and ability to generalize across different datasets. These insights are critical for developing efficient and reliable CNN models in medical image classification tasks, ensuring both high accuracy and computational efficiency.

5 Conclusion and future work

The proposed lightweight Convolutional neural network (CNN) architecture for brain tumor classification achieves excellent performance, reaching an impressive accuracy of 99.54%, while maintaining a compact model size of just 6.89 MB and only 1.8 million parameters. By optimizing key factors such as kernel sizes, max pooling strategies, batch sizes, and cross-dataset validation, the model demonstrates superior classification accuracy, precision, recall, and F1 score. Its shallow architecture ensures computational efficiency and its ability to generalize well across diverse datasets enhances

its potential for MRI Image classification applications. The model's compactness, efficiency, and high performance make it particularly suitable for resource-constrained environments, such as mobile and edge devices, offering a promising solution for automated brain tumor detection in medical imaging systems. This CNN configuration plays a vital role in medical image classification, facilitating more accessible and effective diagnostic tools for brain tumor detection.

Future research could focus on improving the model's generalization further by incorporating advanced data augmentation techniques and exploring more complex architectures, such as residual and attention-based networks to boost accuracy and robustness. Extending the model to handle multi-class classification tasks and testing it on additional medical imaging datasets, such as CT and PET scans, would expand its clinical applicability and versatility. Furthermore, optimizing the model for deployment in real-world environments through techniques like quantization and pruning can improve its efficiency in resource-constrained settings. To ensure seamless integration into clinical workflows, developing user-friendly interfaces and tools for radiologists will be crucial in making the model more accessible and operational. These advancements could significantly enhance the model's impact, enabling real-time, high-accuracy tumor detection across various medical imaging modalities.

Acknowledgements The authors express their gratitude to the National Supercomputing Mission (NSM) for providing the ParamPorul supercomputing resources at the National Institute of Technology Tiruchirappalli, which are essential for meeting the growing computational needs of this research. The National Supercomputing Mission (NSM) is a collaborative initiative managed by the Department of Science and Technology (DST) and the Ministry of Electronics and Information Technology (MeitY), with its implementation spearheaded by the Centre for Development of Advanced Computing (C-DAC), India. We also extend our appreciation to the contributors and curators of the MRI brain tumor datasets used in this research. Additionally, we are grateful to platforms such as Kaggle, GitHub, and Google Colab for offering free computational resources that significantly supported our work.

Author contributions Appasami G. and Nickolas Savarimuthu jointly contributed to the design, implementation, coding, and evaluation of this work. The corresponding author confirms that all authors have reviewed, revised, and approved the final manuscript.

Funding The authors confirm that this research did not receive any specific grant, financial support, or funding from governmental, commercial, or non-profit organizations. The work was carried out using freely available computational and data resources, emphasizing the authors' independence in conducting and reporting this research.

Code availability The code for optimizing kernel and pooling with custom-built CNN models discussed in this research paper is available at the following link: <https://github.com/appasami/Lightweight-CNN-OPT/>

Data availability The complete MRI image data set for training and testing in this research paper is publicly and freely available at the following link: <https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset>

Declarations

Ethical approval and consent to participate This research on the classification of MRI brain tumor images using a CNN model with various kernels and pooling optimization was conducted with publicly available datasets. No human participants or animals were involved in the analysis of data for this research, so ethical approval was not required. The diagnosis and outcome derived from this research work are solely intended for academic understanding and requires further validation and verification for other purpose.

Competing interests The authors declare no competing interests.

Consent for publication This research utilizes publicly available, anonymized MRI brain tumor datasets from platforms like Kaggle, with no direct involvement of human participants, animals, or clinical trials. As the data is pre-collected and openly shared for academic purposes, no ethical approval or informed consent was required. All experiments were computational, ensuring compliance with ethical standards.

Clinical Trial Not Applicable. This study does not qualify as a clinical trial as it exclusively utilizes publicly available MRI brain tumor datasets obtained from Kaggle. The research does not involve human or animal subjects, clinical interventions, experimental treatments, or patient data collection. Instead, it is based on computational experiments conducted using pre-collected and openly shared datasets intended for academic and research purposes. Therefore, clinical trial registration is not applicable to this work.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

- Gong D, Wang X, Liu Z. Deep learning for brain tumor classification: a survey. *Med Image Anal.* 2024;79: 102465. <https://doi.org/10.1016/j.media.2024.102465>.
- Iglewicz B, Smith J, Thompson R. A hybrid deep learning model for brain tumor detection and classification using MRI scans. *J Digit Imaging.* 2024;37(2):182–94. <https://doi.org/10.1007/s10278-024-00525-w>.
- Prados Carrasco F, Vinegoni C, Eduljee A. Inaugural editorial for discover imaging. *Discov Imaging.* 2024;1:1. <https://doi.org/10.1007/s44352-024-00002-x>.
- Aamir M, Namoun A, Munir S, et al. Brain tumor detection and classification using an optimized convolutional neural network. *Diagnostics.* 2024;14(16):1714. <https://doi.org/10.3390/diagnostics14161714>.
- Zhou Z, Li X, Yu H. A lightweight convolutional neural network for brain tumor classification. *Comput Intell Neurosci.* 2024;2024: 305379. <https://doi.org/10.1155/2024/305379>.
- Ahamed NI, Rahman F, Hossain D. Efficient deep learning approach for brain tumor classification from MRI images using a hybrid CNN. *Comput Biol Med.* 2024;152: 106408. <https://doi.org/10.1016/j.combiomed.2024.106408>.
- Singh M, Kumar A. Hyperparameter optimization for CNN models in medical imaging: application to brain tumor classification. *IEEE J Biomed Health Inf.* 2019;23(9):1859–71. <https://doi.org/10.1109/JBHI.2019.2901897>.
- Khan I, Lee H, Hussain T. Brain tumor detection and classification using a hybrid deep learning model. *J Neural Eng.* 2024;21(4): 046004. <https://doi.org/10.1088/1741-2552/abbb72>.
- Wu Z, Sun Y, Zhang L. Multi-scale CNN for brain tumor classification: an experimental study on MR images. *Neural Process Lett.* 2024;60:1–19. <https://doi.org/10.1007/s11063-024-10795-0>.
- Zhang X, Wang F. A hybrid CNN architecture for brain tumor classification using multi-modal MRI data. *J Ambient Intell Human Comput.* 2024;15(1):45–60. <https://doi.org/10.1007/s12652-024-03860-5>.
- Appasami G, Savarimuthu N. Federated learning for secure medical MRI brain tumor image classification. *Eur Phys J Spec Top.* 2025. <https://doi.org/10.1140/epjs/s11734-025-01516-z>.
- Başarslan MS. M-C&M-BL: a novel classification model for brain tumor classification: multi-CNN and multi-BiLSTM. *J Supercomput.* 2025;81:502. <https://doi.org/10.1007/s11227-025-06964-x>.
- Singh D, Patil R, Gupta P. Brain tumor classification using multi-layer convolutional neural networks with feature fusion. *Comput Methods Programs Biomed.* 2024;213: 106471. <https://doi.org/10.1016/j.cmpb.2024.106471>.
- Saeed R, Ibrahim A, Sufi N. MRI-based brain tumor classification with deep learning: a comprehensive review and new perspectives. *IEEE Access.* 2024;12:35658–74. <https://doi.org/10.1109/ACCESS.2024.1234567>.
- Xie L, Zhang Y. Optimizing convolutional neural networks for MRI brain tumor classification. *IEEE Trans Biomed Eng.* 2023;70(2):345–57. <https://doi.org/10.1109/TBME.2022.1234567>.
- Nguyen HT, Lee S. Evaluating the impact of kernel size and pooling layers in convolutional neural networks for MRI tumor diagnosis. *IEEE Access.* 2021;9:23087–100. <https://doi.org/10.1109/ACCESS.2021.3069131>.
- Liu W, Cheng Y. Hybrid deep learning model for brain tumor detection. *Appl Soft Comput.* 2022;119: 107204. <https://doi.org/10.1016/j.asoc.2022.107204>.
- Raza SEA, Shah SA, Khan S. Automated brain tumor classification using deep learning methods on multi-modality MRI scans. *J Med Imaging Health Inform.* 2024;14(2):339–51. <https://doi.org/10.1166/jmihi.2024.10275>.
- Thapliyal A, Verma S, Pandey S. Deep convolutional neural network-based approach for classification of brain tumors from MRI images. *Biol Cybern.* 2024;118(1):39–56. <https://doi.org/10.1007/s00422-023-00942-7>.
- Li J, Zhang L. Cross-dataset performance evaluation of CNN models for brain tumor MRI classification. *J Imaging Sci Technol.* 2020;64(6):156–67. <https://doi.org/10.1007/s11063-020-10555-6>.
- Chen Q, Zhao Y. Cross-dataset evaluation of CNN architectures for brain tumor classification in MRI. *IEEE Trans Image Process.* 2020;29:4761–74. <https://doi.org/10.1109/TIP.2020.2998888>.
- Patel R, Sharma S, Gupta D. Multi-modal CNN for tumor detection. *J Med Syst.* 2021;45(2):119–30. <https://doi.org/10.1007/s10916-021-01795-z>.
- Li C, Zhang F, Du Y, et al. Classification of brain tumor types through MRIs using parallel CNNs and firefly optimization. *Sci Rep.* 2024;14:15057. <https://doi.org/10.1038/s41598-024-65714-w>.
- Patel A, Gupta S. Optimization of CNNs for tumor detection in MRI images: a comparative study. *Comput Intell Neurosci.* 2023;2023:8904563. <https://doi.org/10.1155/2023/8904563>.
- Kumar A, Gupta P, Sharma S, Agarwal R. ResNet50V2 for brain tumor classification. *Int J Comput Vis.* 2021;35(8):599–611. <https://doi.org/10.1007/s11263-021-01442-7>.
- Hassan F, Rahman M. Evaluation of CNN hyperparameters for MRI brain tumor classification using cross-dataset analysis. *J Real-Time Image Process.* 2022;19(2):321–34. <https://doi.org/10.1007/s11554-022-01083-x>.
- Sahoo S, Mishra S, Brahma B, et al. SSO-CCNN: A correlation-based optimized deep CNN for brain tumor classification using sampled PGGAN. *Int J Comput Intell Syst.* 2024;17:179. <https://doi.org/10.1007/s44196-024-00574-w>.
- Sekhar C, Patil S, Goud M. VGG16 for brain tumor classification. *J Digit Imaging.* 2022;35(1):84–98. <https://doi.org/10.1007/s10278-022-00579-w>.
- Saranya SM, Komarasamy D, Dharshini R, et al. Enhancing brain tumor classification with optimized convolutional neural networks. In: *Proceedings of the 14th international conference on computing communication and networking technologies (ICCCNT)*, Delhi, India; 2023. p. 1–6. <https://doi.org/10.1109/ICCCNT56998.2023.10307287>.
- Alturki A, Bin Ali M, El-Baz A. EfficientNetV2B2 for brain tumor detection. *Springer J Med Artif Intell.* 2023;31(4):222–34. <https://doi.org/10.1007/s10115-023-01548-9>.
- Zhang H, Li Y, Luo X. Hybrid CNN-RNN for tumor detection. *Med Image Anal.* 2022;73(7):345–59. <https://doi.org/10.1016/j.media.2022.102425>.

32. Hammad S, Alharbi S, Rizwan M. NASNetMobile for tumor detection. *J Comput Imaging*. 2023;10(5):201–13. <https://doi.org/10.1007/s11761-023-08603-0>.
33. Muthusamy S, Rajan M. CNN-based classification of brain tumors in multi-modal MRI data. *IEEE Trans Med Imaging*. 2022;41(4):1021–33. <https://doi.org/10.1109/TMI.2022.3150221>.
34. Tandel A, Sharma N, Kumar P. ResNet152V2 for tumor classification. *J Neuroimaging*. 2021;11(3):312–21. <https://doi.org/10.1002/jon.25867>.
35. Kurdi SZ, Ali MH, Jaber MM, et al. Brain tumor classification using meta-heuristic optimized convolutional neural networks. *J Pers Med*. 2023;13(2):181. <https://doi.org/10.3390/jpm13020181>.
36. Ravi M, Srinivasan M. Optimizing convolutional neural networks for efficient brain tumor detection in MRI. *J Comput Biol*. 2021;28(3):256–69. <https://doi.org/10.1007/s10862-021-09843-x>.
37. Ghassemi M, Rezaei N, Azizi M. InceptionV3 for brain tumor classification. *J Artif Intell Med*. 2020;45(9):123–35. <https://doi.org/10.1007/s10366-020-00573-4>.
38. Liu J, Zhang Y, Chen Z, Wang Q. ResNet101V2 for tumor detection. *IEEE Trans Biomed Eng*. 2023;70(2):532–40. <https://doi.org/10.1109/TBME.2023.3080905>.
39. Sasidhar S, Meena P, Rani S. VGG19 for brain tumor detection. *Comput Imaging Vis*. 2023;58(6):457–69. <https://doi.org/10.1007/s10667-023-00420-7>.
40. Chen L, Xiao W. Hybrid CNN-LSTM model for brain tumor classification. *Neurocomputing*. 2021;474:467–76. <https://doi.org/10.1016/j.neucom.2020.10.066>.
41. Hossain MA, Alam MS, Rahman MM, Alam MS. Deep learning models for brain tumor detection. *J Med Imaging*. 2023;12(4):234–45. <https://doi.org/10.1007/s00134-023-07056-2>.
42. Swati S, Patel A, Prakash N. MobileNet for brain tumor detection. *J Imaging Sci Technol*. 2019;27(3):102–15. <https://doi.org/10.1117/1.JIST.27.3.102>.
43. Alanazi A, Alzahrani F, Khamis M. DenseNet201 for brain tumor classification. *Neural Process Lett*. 2022;54(2):105–20. <https://doi.org/10.1007/s11063-022-10602-w>.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.